

Huawei ICT Competition 2025-2026
Regional Stage
Computing Track
Lab Exam



HUAWEI

Huawei Technologies Co., LTD.

All Rights Reserved.

Contents

- 1 Background of Task Design3**
- 2 Exam Description.....5**
 - 2.1 Weighting 5
 - 2.2 Exam Requirements..... 5
 - 2.3 Exam Platform..... 5
 - 2.4 Saving Tasks..... 6
- 3 Exam Questions7**
 - 3.1 openEuler Tasks 7
 - 3.1.1 Scenario 7
 - 3.1.2 Network Topology 7
 - 3.1.3 Exam Resources 8
 - 3.1.4 Exam Requirements 10
 - 3.1.5 Exam Tasks..... 10
 - 3.2 openGauss Tasks 15
 - 3.2.1 Scenario 15
 - 3.2.2 Network Topology 16
 - 3.2.3 Exam Resources 16
 - 3.2.4 Exam Tasks..... 17
 - 3.3 Kunpeng Application Development Tasks..... 29
 - 3.3.1 Scenario 29
 - 3.3.2 Network Topology 29
 - 3.3.3 Exam Resources 29
 - 3.3.4 Exam Tasks..... 30

1

Background of Task Design

As digital transformation accelerates, enterprises and organizations need to build more efficient, secure, and intelligent information systems to cope with increasingly complex and changeable service requirements. To address this issue, Huawei has launched a computing platform based on its self-developed Kunpeng processors. The platform provides a high-performance, reliable, and secure hardware foundation, along with a rich software ecosystem. It supports a wide range of application scenarios, including servers, cloud computing, edge computing, and embedded systems.

The openEuler OS is a fundamental component of the Kunpeng computing platform, and it is important that those users have a strong understanding of openEuler, including how to use the command-line interface (CLI) and scripts to interact with the system, as well as how to deploy services and clusters on openEuler servers. This can include how to deploy a blog website cluster based on the Linux-Apache-MySQL-PHP (LAMP) stack, for example. This cluster supports domain name resolution (DNS) for users to access the website using a domain name. The frontend of the website is a Linux Virtual Server (LVS) cluster with Keepalived to implement high availability (HA) load balancing. A network address translation (NAT) gateway is set up to ensure the blog website security and save cloud resources. Ansible is used to install and deploy the many hosts in the overall architecture. Therefore, it is important that partners aiming to co-construct the Kunpeng ecosystem possess the ability to build enterprise-class services and cluster architectures on openEuler.

As openGauss forms a crucial component of the Kunpeng computing platform, and databases are essential software for enterprises, computing professionals must acquire specific, necessary skills, such as SQL operations, complex statement queries, basic database operations and maintenance, and database application development (such as stored procedures and user-defined functions). Additionally, with the aid of AI technologies, openGauss can leverage AI capabilities to enhance database operations and performance tuning. Therefore, acquiring and enhancing these skills can empower openGauss practitioners to leverage databases better, thereby enhancing work quality and efficiency.

Kunpeng application development is a process that leverages the power of the Kunpeng computing platform. This process involves porting, profiling, tuning, deploying, and releasing applications. Developers that will develop applications on Kunpeng need to have a thorough understanding of the features and advantages of the Kunpeng computing platform. They should be adept at utilizing the tools and services provided by the platform to achieve swift porting, efficient running, and easy release of applications. Code porting is often a complex and tedious process. Developers are required to analyze and inspect software packages, source code, and dependency libraries, and address differences between instruction sets, such as variations in syntax, instructions, functions, and library support. Faced with porting challenges like laborious manual code review, demanding expertise, repetitive compilation/troubleshooting cycles, and overall inefficiency, the Porting Advisor can provide professional porting guidance based on quick and automatic scan of massive code. Becoming proficient in Kunpeng application development is crucial for being able to confidently work on the Kunpeng computing platform.

In short, this exam is designed to examine candidates' basic knowledge and capabilities in application development and O&M on the Kunpeng computing platform, including the openEuler OS, openGauss database, and Kunpeng application development. The three aspects cover OS O&M, basic software deployment, data processing, data management, database development, and Kunpeng code porting.

2 Exam Description

2.1 Weighting

The exam consists of three parts: openEuler, openGauss, and Kunpeng Application Development. The maximum number of points in this exam is 1,000 points.

Domain	Weight	Points
openEuler	50%	500
openGauss	30%	300
Kunpeng Application Development	20%	200

2.2 Exam Requirements

- Read the *Exam Guide* and exam tasks carefully before taking the exam.
- If multiple solutions are available for a task, select the one you think is best.
- Set passwords for resources used in the exam.

Note: Follow the instructions as written to avoid being penalized.

2.3 Exam Platform

The lab exam environment is Huawei Cloud: <https://www.huaweicloud.com/intl/en-us/>

- Log in using an account that has applied for a coupon. For details, see the *Exam Guide*.
- You need to create the cloud resources required by the exam. When purchasing resources, select **pay-per-use** billing.
- If resources of a required flavor are sold out, purchase resources of a similar flavor.

2.4 Saving Tasks

All answers must be recorded as **screenshots** as instructed. Read the *Exam Guide* for details about result submission.

3

Exam Questions

3.1 openEuler Tasks

3.1.1 Scenario

Your proficiency at using openEuler will be examined in the following aspects:

System and service management: This focuses on the core O&M tasks on openEuler. All services rely on a stable and correctly configured system environment. Your proficiency in system service lifecycle management, key configuration modification, and web service (such as Nginx) deployment and management will be examined.

Resource monitoring and performance analysis: To keep services stable and performing at their best, it is essential to maintain deep insight into the system's operational status. Analyzing key metrics such as the CPU, memory, and network by using resource monitoring tools is a precondition for performance tuning and troubleshooting.

Storage and cluster configuration: To cope with complex production scenarios and implement data sharing and service high availability (HA), you need to master how to configure and manage network shared storage (such as NFS) and basic cluster environments.

Application of container technologies: Containerization is the standard form of modern application deployment. This exam assesses your ability to use container engines such as iSula. More specifically, how you build images, and run and manage containers for rapid application deployment.

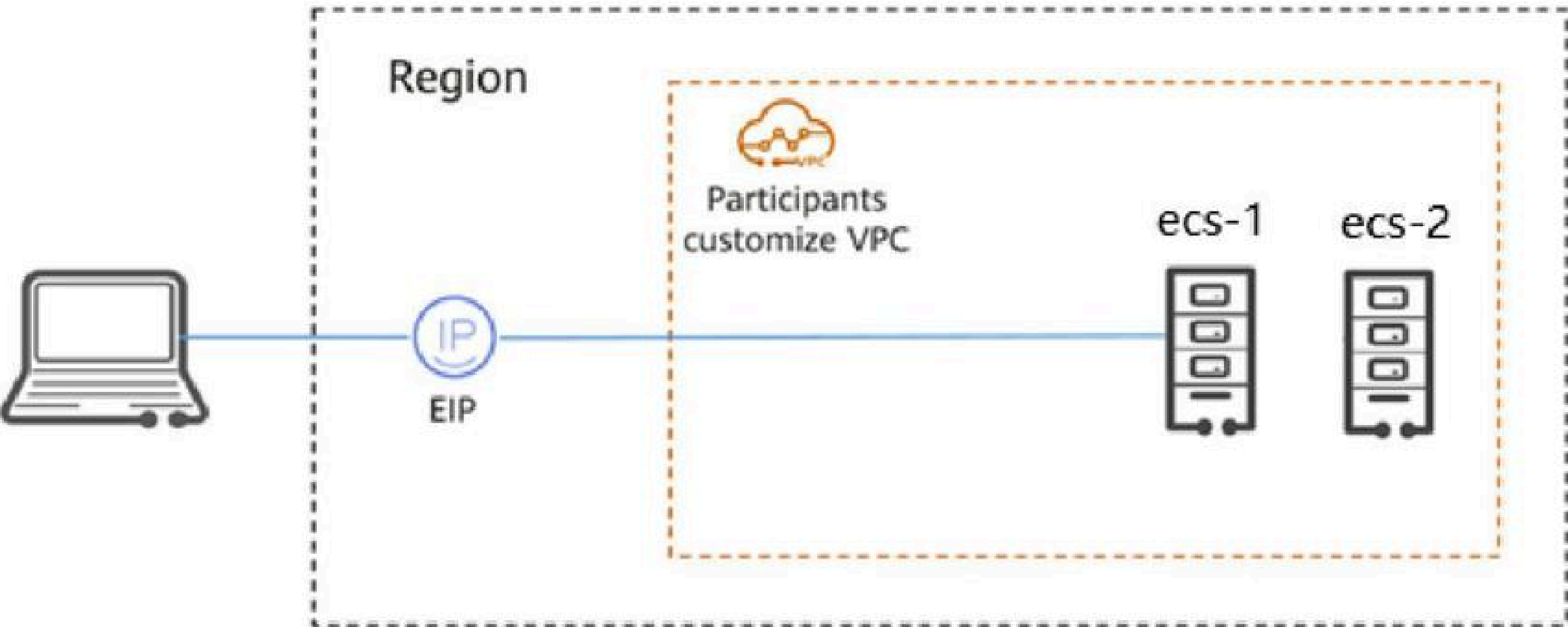
Container cluster and load balancing practice: This task tests your comprehensive system capabilities. Currently, a large number of network services are shifting from traditional architectures to containerized cluster deployments. Mastering the skills of building service clusters based on containers and implementing load balancing and HA through components such as Nginx is essential.

3.1.2 Network Topology

Description:

You need to purchase two Elastic Cloud Servers (ECSs) in the same subnet on Huawei Cloud and name them **ecs-1** and **ecs-2**. (To avoid network address conflicts, you need to create your own VPC.) **ecs-1** functions as the active server, and **ecs-2** functions as the standby server. Bind elastic IP addresses (EIPs) to the ECSs for login in subsequent tasks. The following table lists the private IP address and recommended purpose for each ECS.

ECS	Private IP Address	Recommended Purpose
ecs-1	x.x.0.101 (for example, 192.168.0.101/24)	Tasks 1, 2, 3, and 4
ecs-2	x.x.0.102 (for example, 192.168.0.102/24)	Tasks 1, 2, and 4



Notes:

When modifying configuration files, you are advised to back up the original files immediately. This allows for easy restoration of environments if you need to reuse the ECSs. You can decide whether to reset the environments during the exam. Save the screenshots of the completed tasks if you need to delete an existing ECS and purchase a new one.

3.1.3 Exam Resources

3.1.3.1 Lab Environment

Set up the Huawei Cloud environment in accordance with the exam requirements. For accounts created on the Huawei Cloud international website, the CN-Hong Kong or AP-Singapore region is recommended. Your computer must have Internet access to complete the tasks in this exam.

3.1.3.2 Cloud Resources

Resource	Specifications	Description
VPC	None	Custom VPC

ECS	4 vCPUs 8 GB openEuler 22.03 64bit with ARM Disk capacity: 40 GB	ecs-1
ECS	4 vCPUs 8 GB openEuler 22.03 64bit with ARM Disk capacity: 40 GB	ecs-2
EIP and bandwidth	Set it as required by the tasks.	
EIP and bandwidth	Set it as required by the tasks.	

Configure the ECSs as follows:

Billing Mode: Pay-per-use

CPU Architecture: Kunpeng

Specifications: Kunpeng general computing-plus | kc1.xlarge.2 | 4 vCPUs | 8 GiB

Image: openEuler 22.03 64bit with ARM (40GiB) under Public image

System Disk: 40 GiB

Data Disk: No data disk is required.

Network: Configure the VPC and manually specify the IP addresses as required by the network topology.

Security Group: Configure it as required.

Billed By: Traffic

Bandwidth Size: 50 Mbit/s

ECS Name: Set the names as required by the tasks.

Password: Set it as required.

Enterprise Project: default

Resource release: After completing the exam, you need to release all resources, including the ECSs, EIPs, system disks, data disks, and custom VPCs.

3.1.3.3 Tools

The following table lists the tool required in this exam.

Software Package	Description
MobaXterm	Remote login tool

3.1.4 Exam Requirements

- Take screenshots for all key steps and save them in the .png format. Name screenshots as required by the exam tasks.
- Strictly follow the configuration and naming requirements described in the exam.
- MobaXterm is recommended for logging in to openEuler.
- Each step in a task is scored separately. Please plan your time appropriately.

3.1.5 Exam Tasks

Task 1: Building a Container Node (100 points)

Subtask 1: Upgrade the OS kernel of the container node.

Procedure:

Note: The iSula network feature involved in this exam task is supported only in openEuler 24.03 LTS SP2. You need to upgrade the OS kernel to ensure that network functions can be used properly.

- a. Use **ecs-1** as the container node and back up the Yum repository of the host. The backup file name extension is **.bak**.
- b. Replace **22.03-LTS** in the Yum repository with **24.03-LTS-SP2**.
- c. Run **uname -a** to check the current OS version and kernel.
- d. Use the DNF utility to upgrade the OS version.

Note: Run the following command to upgrade the OS.

```
dnf -y update --releasever='24.03LTS-SP2' --allowdowngrading --skip-broken
```

- e. Restart **ecs-1** and verify the OS kernel version upgrade.

Screenshot requirements:

- a. Take a screenshot of the Yum repository backup on **ecs-1**, and save it as **1-1-1backup**.
- b. Take a full screenshot of the Yum repository version update, and save it as **1-1-2yum**.
- c. Take a full screenshot of the current OS kernel version, and save it as **1-1-3uname1**.
- d. Take a screenshot of the last part of the OS version update command output, and save it as **1-1-4kernel**.
- e. Take a full screenshot of the OS kernel version after the host is restarted, and save it as **1-1-5uname2**.

Subtask 2: Build a container node.

Procedure:

- a. Install the iSula container software and set it to automatically start upon system startup.

- b. Change the iSula container image repository to **<https://docker.m.daocloud.io>** and restart the iSula container service.
- c. Pull the Nginx image and verify the image pulling result.
- d. Install the Git utility to obtain the iSula network plugin CNI.
Plugin address: https://gitee.com/yftyxa/cni_for_i-sulad_arm.git
- e. Create a plugin directory **/opt/bin/cni** and decompress the plugin to this directory.
- f. Modify the iSula configuration file to configure the network plugin and restart the iSula service. The template is as follows:

```
"network-plugin": "cni",  
"cni-bin-dir": "/opt/bin/cni",  
"cni-conf-dir": "/etc/cni/net.d"
```

Screenshot requirements:

- a. Take a screenshot of installing the software on **ecs-1** and setting the software to automatically start upon system startup, and save it as **1-2-1isula**.
- b. Take a full screenshot when modifying the container image repository and restarting the container service, and save it as **1-2-2repo**.
- c. Take a full screenshot when pulling the Nginx image and verifying the image, and save it as **1-2-3pull**.
- d. Take a full screenshot when installing the Git utility and obtaining the plugin, and save it as **1-2-4git**.
- e. Take a full screenshot when creating the plugin directory and decompressing the plugin to the directory, and save it as **1-2-5cni**.
- f. Take a full screenshot when configuring the container configuration file and restarting the container service, and save it as **1-2-6config**.

Task 2: Constructing NFS Shared Storage (100 points)

Subtask 1: Deploy the NFS shared storage server.

Procedure:

- a. Create a shared directory **/nfs/share** on the shared storage node **ecs-2**.
- b. Create a shared file named **shared.html** in the shared directory and write "This is shared data" to the file.
- c. Configure the NFS export rule and add the rule to the NFS configuration file **/etc/exports**.

Requirements:

1. Only the **ecs-1** host is allowed to access the shared directory.

2. Access permissions: Read/Write (RW), Asynchronous (Async), allowing root access (with no root squash).
- d. Start the nfs-server service and set it to automatically start upon system startup.
- e. On **ecs-1**, verify and display the current NFS export list to check whether the share configuration takes effect.

Screenshot requirements:

- a. Take a screenshot when creating the shared directory on **ecs-2**, and save it as **2-1-1share**.
- b. Take a full screenshot when creating the shared file in the shared directory, and save it as **2-1-2shared-data**.
- c. Take a full screenshot of the configuration rule, and save it as **2-1-3rule**.
- d. Take a full screenshot of the nfs-server startup and automatic startup configuration, and save it as **2-1-4start**.
- e. Take a full screenshot of the NFS export list, and save it as **2-1-5export**.

Subtask 2: Deploy the NFS shared storage client.

Procedure:

- a. On the container node **ecs-1**, create a client directory **/nfs/client** and mount the shared directory of **ecs-2** to this directory.
- b. On the container node **ecs-1**, access the **shared.html** file in the shared directory to check whether the shared storage takes effect.

Screenshot requirements:

- a. Take a screenshot when creating an NFS client directory and mounting the shared directory to the client directory, and save it as **2-2-1mount**.
- b. Take a full screenshot when accessing the shared file in the shared directory, and save it as **2-2-2share**.

Task 3: Deploying the Nginx Service Container (100 points)

Subtask 1: Deploy differentiated files.

Procedure:

- a. Create the **web1** and **web2** directories in the shared directory on the shared storage node **ecs-2**.
- b. Create an **index.html** file in the **web1** and **web2** directories and write "This is webnode1" and "This is webnode2" to the files, respectively.
- c. On the container node **ecs-1**, access the **index.html** files in the **web1** and **web2** directories in the shared directory and verify the access result.

Screenshot requirements:

- a. Take a screenshot of the created directory, and save it as **3-1-1webdir**.
- b. Take a screenshot of the HTML files created in the directories and the file content, and save it as **3-1-2index**.
- c. Take a full screenshot of the HTML files in the shared directories, and save it as **3-1-3verify**.

Subtask 2: Deploy a container cluster.

Procedure:

- a. Create a container network **cni-nginx** on the container node **ecs-1**, and view the network information.
- b. Deploy the first Nginx service container as follows:
 - (a) Use the container network **cni-nginx**.
 - (b) Map port 80 listened by the Nginx container to port 8081 of the host machine.
 - (c) Mount the **web1** directory in the shared directory to the corresponding directory of the Nginx container.
 - (d) Run **nginx -g "daemon off;"** to keep the container running in the foreground.
- c. Deploy the second Nginx service container as follows:
 - (a) Use the container network **cni-nginx**.
 - (b) Map port 80 listened by the Nginx container to port 8082 of the host machine.
 - (c) Mount the **web2** directory in the shared directory to the corresponding directory of the Nginx container.
 - (d) Run **nginx -g "daemon off;"** to keep the container running in the foreground.
- d. Check the running status of both containers and ensure that the containers are in the up state.
- e. Run **curl** to access **index.html** on ports 8081 and 8082, respectively, and check whether the access is normal.

Screenshot requirements:

- a. Take a screenshot when creating the container network and viewing the network information, and save it as **3-2-1cni**.
- b. Take a screenshot of the first container, and save it as **3-2-2nginx1**.
- c. Take a screenshot of the second container, and save it as **3-2-3nginx2**.
- d. Take a screenshot showing that the containers are running properly, and save it as **3-2-4verify**.
- e. Take screenshots of the two ports accessed using the **curl** command, and save them as **3-2-5curl1** and **3-2-6curl2**.

Task 4: Deploying Load Balancing (100 points)

Subtask: Deploy a load balancer.

Procedure:

- a. On the container node **ecs-1**, create the load balancer configuration file **/tmp/nginx-lb.conf**, add ports 8081 and 8082 to the backend server group, and listen to port 8080.
- b. Deploy the Nginx load balancing container using iSula as follows:
 - (a) Use **--net host** to solve network isolation problems.
 - (b) Mount the load balancer configuration file on the host to the corresponding configuration file directory of the container.
 - (c) Run **nginx -g "daemon off;"** to keep the container running.
- c. Check whether the load balancer is running properly.

Screenshot requirements:

- a. Take a screenshot of the load balancer configuration file, and save it as **4-1-1lb-conf**.
- b. Take a screenshot of the load balancer deployment and verification, and save it as **4-1-2lb-nginx**.
- c. Take a screenshot of the load balancer running properly, and save it as **4-1-3verify**.

Task 5: System Monitoring and Comprehensive Verification (100 points)

Subtask 1: Monitor system resources.

Procedure:

- a. Run **top** to check the system resources of the container node and retain the first 20 lines.
- b. Run **free** to check the system memory usage of the container node.
- c. Run **df** to check the system drive usage of the container node.
- d. Run **ss** to check the usage of port 80 on the container node.

Screenshot requirements:

- a. Take a full screenshot of the system resources on the node, and save it as **5-1-1top**.
- b. Take a full screenshot of the system memory usage on the node, and save it as **5-1-2free**.
- c. Take a full screenshot of the system drive usage on the node, and save it as **5-1-3df**.
- d. Take a full screenshot of the 80 port usage on the node, and save it as **5-1-4port-80**.

Subtask 2: Verify container cluster resources.

Procedure:

- a. Use a **for** loop and the **curl** command to access **index.html** on port 8080 of the container node 10 times, to verify the effectiveness of the load balancer.

Screenshot requirements:

- a. Take a screenshot when accessing the load balancer 10 times, and save it as **5-2-1verify-lb**.

Subtask 3: Adjust the load balancing policy.

Procedure:

- a. Delete the load balancing container.
- b. Modify the load balancer configuration file. Set the weight of port 8081 to 99 and the weight of port 8082 to 1.
- c. Create a load balancing container as follows:
 - (a) Use **--net host** to solve network isolation problems.
 - (b) Mount the modified load balancer configuration file to the container configuration file **conf.d/load-balancer.conf**.
 - (c) Run **nginx -g "daemon off;"** to keep the container running.
- d. Use a **for** loop and the **curl** command to access **index.html** on port 8080 of the container node 10 times, to verify the effectiveness of the load balancer.

Screenshot requirements:

- a. Take a screenshot when deleting the load balancing container, and save it as **5-3-1delete**.
- b. Take a full screenshot of the modified configuration file, and save the screenshot as **5-3-2weight**.
- c. Take a screenshot when creating the load balancing container, and save it as **5-3-3newlb**.
- d. Take a screenshot of the load balancer running properly, and save it as **5-3-4verify**.

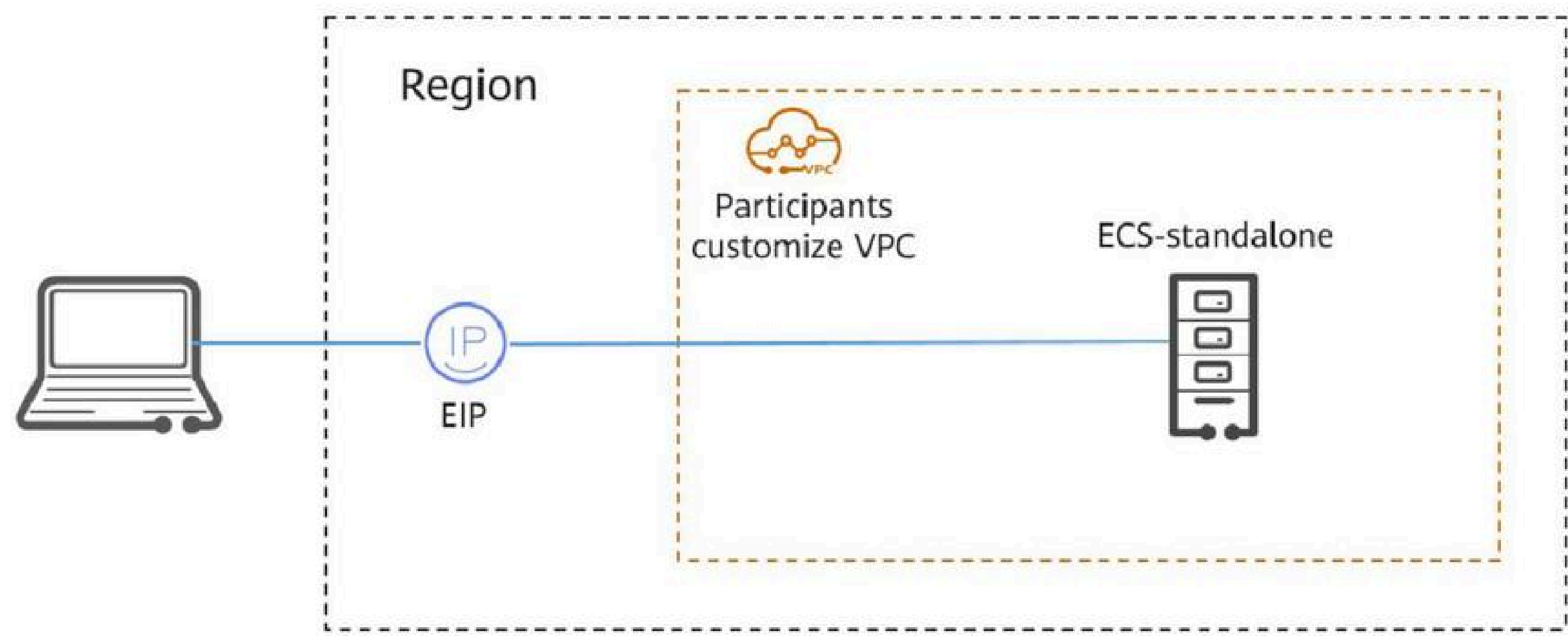
3.2 openGauss Tasks

3.2.1 Scenario

A manufacturing company uses openGauss to manage supplier information, inventory, supply orders, and logistics data. Its core requirements include creating standard table structures, implementing batch data migration, developing service warning functions, managing data access permissions, optimizing query performance, and configuring O&M audit to ensure the integrity, security, and availability of supply chain data.

3.2.2 Network Topology

Description: The following figure shows the network topology of this exam. You can define a VPC or use the default VPC.



Introduction:

To install and deploy openGauss (version 5.0.0), you'll need to purchase an elastic cloud server (ECS) in the Huawei Cloud. The hostname must be **ict-opengauss**, the ip must be **192.168.0.228** . Additionally, you'll need to purchase an elastic IP address (EIP) and bind it to the server before establishing a connection to the server from your computer. After configuration, you will connect to the server from your local computer via this public IP address.

3.2.3 Exam Resources

3.2.3.1 Lab Environment

Set up the Huawei Cloud environment in accordance with the exam requirements. For accounts created on the Huawei Cloud international website, the CN-Hong Kong or AP-Singapore region is recommended. Your computer must have Internet access to complete the tasks in this exam.

3.2.3.2 Cloud Resources

Resource Name	Specifications	Description
VPC	None	vpc-default
ECS	8vcpus 16GB openEuler 22.03 with ARM 64bit Disk capacity: 60 GB	You have to name the ECS ict-opengauss IP address: 192.168.0.228
EIP and bandwidth	As required by tasks	
IMS	Shared image name: ict-opengauss	

3.2.3.3 Tools

Software Package	Description
MobaXterm	Remote login tool

3.2.4 Exam Tasks

Task 1: Setting Up the Supply Chain Database Environment (57 Points)

Subtask 1: Create a tablespace `ts_supplychain`.

Environment preparation:

Create a Kunpeng ECS in a specified region.

Note: Please create the ECS strictly in accordance with the "Cloud Resources" description. You need to select the shared `ict-opengauss` image and the fixed IP address `192.168.0.228`. Choosing other options, the following subtasks cannot be completed. For details about ECS specifications requirements, see the table in "Cloud Resources."

Run the following commands to create a directory:

```
mkdir -p /opt/opengauss/tablespaces/ts_supplychain
chown -R omm:dbgrp /opt/opengauss/tablespaces/ts_supplychain
chmod 700 /opt/opengauss/tablespaces/ts_supplychain
```

Procedure:

- a. Log in to the **postgres** database (port: **26000**) as the **omm** user, create an absolute-path directory for the tablespace, and run the SQL command to create the tablespace. The absolute-path directory is **/opt/opengauss/tablespaces/ts_supplychain**.

Screenshot requirements:

- a. Take a screenshot of the SQL command for creating the tablespace and the verification result by running the `\db` command, and save it as **1-1-1 create_ts**.

Subtask 2: Create a database `db_supplychain`.

Procedure:

- a. Log in to the **postgres** database as the **omm** user, create the **db_supplychain** database whose code set is **UTF8** and tablespace is **ts_supplychain**, and run the meta command to switch to the database.

Screenshot requirements:

- a. Take a screenshot of the SQL commands for creating and switching to the database and the execution result, and save it as **1-2-1 create_change_db**.

Subtask 3: Create core tables in the supply chain database.

Procedure:

- a. Create four core tables in the **db_supplychain** database, with PRIMARY KEY, FOREIGN KEY, and other necessary constraints configured.
- b. Run the \d command to verify the result.

The table structure is as follows:

Table Name (Remarks)	Column (Constraint)	Data Type
supplier (supplier table)	s_suppkey (PRIMARY KEY) s_name (NOT NULL) s_nationkey (NOT NULL) s_phone (NOT NULL) s_comment	INT VARCHAR(50) INT VARCHAR(20) VARCHAR(200)
part (part table)	p_partkey (PRIMARY KEY) p_brand (NOT NULL) p_type (NOT NULL) p_size (NOT NULL) p_container (NOT NULL) p_comment	INT VARCHAR(30) VARCHAR(50) INT VARCHAR(20) VARCHAR(200)
supply_order (supply order table)	so_orderkey (PRIMARY KEY) so_suppkey (NOT NULL, FOREIGN KEY referencing supplier.s_suppkey) so_partkey (NOT NULL, FOREIGN KEY referencing part.p_partkey) so_quantity (NOT NULL, CHECK (greater than 0)) so_supplydate (supply date, NOT NULL)	The data type of so_supplydate is DATE, and that of other columns is INT.
logistics (logistics table)	l_id (PRIMARY KEY) l_so_key (NOT NULL, FOREIGN KEY referencing supply_order.so_orderkey) l_shipmode (transportation mode, NOT NULL) l_status (logistics status, NOT NULL)	l_id and l_so_key are of the INT type, l_shipmode is of the VARCHAR(20) type, and l_status is of the VARCHAR(10) type.

Screenshot requirements:

- Take a screenshot of all table creation SQL statements and execution results, and save it as **1-3-1 create_tables**. The column constraints must be met. Otherwise, no point is given.
- Take a screenshot of the `\d` command output, and save it as **1-3-2 table_list**. The screenshot must contain the current database name and the command output indicating that the tables are created successfully.

Subtask 4: Configure a partitioned table for the `supply_order` table.

Procedure:

- Change the `supply_order` table to a range partitioned table by supply date (`so_supplydate`). Create a partition for each quarter in 2024 (for example, 2024-01-01 to 2024-03-31 is the Q1 partition). Rebuild the partitioned table based on the current table columns. Make sure to screenshot the process.
- Run the `\d+ supply_order` command to verify the result.

Screenshot requirements:

- Take a screenshot of the SQL statements for creating the partitioned table based on the current table columns and the table creation result, and save it as **1-4-1 partition_table**.
- Take a screenshot of the `\d+ supply_order` command output, and save it as **1-4-2 partition_table**.

Task 2: Supply Chain Data Migration and Basic Service Query (65 Points)

Environment preparation:

Run the following command:

```
create database test_supply_db;
```

Then, switch to the new database `test_supply_db` and run the following commands:

-- Create the `region` table (region).

```
CREATE TABLE region (  
    r_regionkey INT PRIMARY KEY,  
    r_name VARCHAR(25) NOT NULL UNIQUE,  
    r_comment VARCHAR(152)  
);  
INSERT INTO region VALUES  
(1, 'ASIA', 'Asia, including China and Japan'),  
(2, 'AMERICA', 'America, including the USA and Canada'),  
(3, 'EUROPE', 'Europe, including Germany and France'),  
(4, 'AFRICA', 'Africa, including South Africa and Nigeria'),  
(5, 'MIDDLE EAST', 'Middle East, including Saudi Arabia and the UAE');
```


-- Create the **nation** table (country).

```
CREATE TABLE nation (  
    n_nationkey INT PRIMARY KEY,  
    n_name VARCHAR(25) NOT NULL,  
    n_regionkey INT NOT NULL REFERENCES region(r_regionkey),  
    n_comment VARCHAR(152)  
);  
INSERT INTO nation VALUES  
(1, 'CHINA', 1, 'China, major suppliers in East China'),  
(2, 'JAPAN', 1, 'Japan, mainly electronic part suppliers'),  
(3, 'USA', 2, 'USA, core supplier in North America'),  
(4, 'GERMANY', 3, 'Germany, precision part supplier'),  
(5, 'INDIA', 1, 'India, textile part supplier');
```

-- Create the **supplier** table (supplier).

```
CREATE TABLE supplier (  
    s_suppkey INT PRIMARY KEY,  
    s_name VARCHAR(50) NOT NULL,  
    s_nationkey INT NOT NULL REFERENCES nation(n_nationkey),  
    s_phone VARCHAR(20) NOT NULL,  
    s_comment VARCHAR(200)  
);  
INSERT INTO supplier VALUES  
(101, 'Huawei Supply', 1, '13800138000', 'Huawei supply chain subsidiary, mainly engaged in electronic parts'),  
(102, 'Sony Japan', 2, '81-3-12345678', 'Sony Japan, mainly engaged in optical parts'),  
(103, 'Tesla USA', 3, '1-800-123-4567', 'Tesla USA, mainly engaged in automotive parts'),  
(104, 'Siemens Germany', 4, '49-89-12345678', 'Siemens Germany, mainly engaged in industrial parts'),  
(105, 'Tata India', 5, '91-12-34567890', 'Tata India, mainly engaged in textile parts');
```

-- Create the **part** table (part).

```
CREATE TABLE part (  
    p_partkey INT PRIMARY KEY,  
    p_brand VARCHAR(30) NOT NULL,  
    p_type VARCHAR(50) NOT NULL,  
    p_size INT NOT NULL,  
    p_container VARCHAR(20) NOT NULL,  
    p_comment VARCHAR(200)
```



```
);
INSERT INTO part VALUES
(201, 'Brand#43', 'LARGE METAL', 50, 'BOX', 'Large metal parts for machinery
manufacturing'),
(202, 'Brand#51', 'MEDIUM PLASTIC', 30, 'BAG', 'Medium-sized plastic parts for
electronic equipment'),
(203, 'Brand#43', 'SMALL WOOD', 10, 'CRATE', 'Small wooden parts for packaging'),
(204, 'Brand#22', 'LARGE GLASS', 60, 'CASE', 'Large glass parts for optical equipment'),
(205, 'Brand#43', 'MEDIUM STEEL', 40, 'DRUM', 'Medium-sized steel parts for
building');
```

-- Create the **partsupp** table (supplier-part association).

```
CREATE TABLE partsupp (
    ps_partkey INT NOT NULL REFERENCES part(p_partkey),
    ps_suppkey INT NOT NULL REFERENCES supplier(s_suppkey),
    ps_availqty INT NOT NULL,
    ps_supplycost NUMERIC(10,2) NOT NULL,
    ps_comment VARCHAR(200),
    PRIMARY KEY (ps_partkey, ps_suppkey)
);
INSERT INTO partsupp VALUES
(201, 101, 150, 25.50, 'The inventory of Brand#43 metal parts supplied by Huawei is
sufficient.'),
(201, 102, 80, 26.80, 'The inventory of Brand#43 metal parts supplied by Sony is less
than 100 (warning).'),
(203, 105, 95, 12.30, 'The inventory of Brand#43 wooden parts supplied by Tata is less
than 100 (warning).'),
(205, 104, 200, 38.90, 'The inventory of Brand#43 steel parts supplied by Siemens is
sufficient.'),
(202, 103, 120, 18.70, 'The inventory of Brand#51 plastic parts supplied by Tesla is
sufficient.');
```

-- Create the **supply_order** table (supply order, partitioned table).

```
CREATE TABLE supply_order (
    so_orderkey INT NOT NULL,
    so_suppkey INT NOT NULL REFERENCES supplier(s_suppkey),
    so_partkey INT NOT NULL REFERENCES part(p_partkey),
    so_quantity INT NOT NULL CHECK (so_quantity > 0),
    so_supplydate DATE NOT NULL,
    PRIMARY KEY (so_orderkey, so_supplydate)
) PARTITION BY RANGE (so_supplydate)
(
```



```
PARTITION so_q1_2024 VALUES LESS THAN ('2024-04-01'),
PARTITION so_q2_2024 VALUES LESS THAN ('2024-07-01'),
PARTITION so_q3_2024 VALUES LESS THAN ('2024-10-01'),
PARTITION so_q4_2024 VALUES LESS THAN ('2025-01-01')
);
INSERT INTO supply_order VALUES
(3001, 101, 201, 50, '2024-02-15'),
(3002, 102, 201, 30, '2024-03-20'),
(3003, 105, 203, 40, '2024-05-10'),
(3004, 103, 202, 20, '2024-01-25'),
(3005, 104, 205, 60, '2024-06-30');
```

Subtask 1: Copy and authorize data files.

Procedure:

- Use `gs_dump` to export the tables in the **test_supply_db** database (port: **26000**) to **/home/omm/supply_data.tar**.
- Change the file owner to **omm**, the user group to **dbgrp**, and the permission to **755**, and verify the changes.

Screenshot requirements:

- Take a screenshot of the export result using `gs_dump` and the `ll /home/omm` command output, and save it as **2-1-1-dump**.
- Take a screenshot of the commands and output for changing the file owner to **omm**, the owner group to **dbgrp**, and the permission to **755**, and for verifying the changes. Save it as **2-1-2-change**.

Subtask 2: Use `gs_restore` to import data.

Procedure:

- Run the `gs_restore` command as the **omm** user to import **/home/omm/supply_data.tar** to the **db_supplychain** database (port: **26000**).
- Run the `SELECT COUNT (*) FROM supplier;` command.

Screenshot requirements:

- Take a screenshot of the `gs_restore` command and output, and save it as **2-2-1 gs_restore**.
- Take a screenshot of logging in to the **db_supplychain** database and executing the `SELECT COUNT(*) FROM supplier;` statement, and save it as **2-2-2 data_verify**.

Subtask 3: Use SQL statements to query basic services.

Procedure:

- In the **db_supplychain** database, collect statistics on the total supply amount of each supplier in the **ASIA** region in 2024 (Total supply amount = **so_quantity** × Wholesale

part price, where the wholesale part price is associated with **ps_supplycost** in the **partsupp** table).

- b. Query all suppliers of the Brand#43 parts (displaying **s_name**, **s_phone**, and **s_nationkey**).

Screenshot requirements:

- a. Take a screenshot of the SQL statement and output, and save it as **2-3-1 query_sql**.
- b. Take a screenshot of the SQL statement and output, and save it as **2-3-2 asia_supply_result**.

Subtask 4: Develop a stored procedure for inventory warning.

Procedure:

- a. In the **db_supplychain** database, write a stored procedure **proc_supply_warning** to implement the following function: When a supplier's part inventory amount (**partsupp.ps_availqty**) is less than 100, the warning message "Inventory warning: The inventory amount of % (parts) from % (supplier) is % (inventory amount), which is below the recommended amount." is displayed.
- b. Call the stored procedure and view the result.

Screenshot requirements:

- a. Take a screenshot of the SQL statement for creating the stored procedure, and save it as **2-4-1 proc_create**.
- b. Take a screenshot of the command for calling the stored procedure and the warning output, and save it as **2-4-2 proc_call**.

Task 3: Supply Chain Database Permissions and Security (65 Points)

Subtask 1: Create a service role.

Procedure:

- a. Create a role in the **postgres** database.

role_supply_admin: has all permissions (CREATE, SELECT, INSERT, UPDATE, and DELETE) on the **db_supplychain** database.

Screenshot requirements:

- a. Take a screenshot of the SQL statements for creating and authorizing the **role_supply_admin** role, and save it as **3-1-1 role_admin**.

Subtask 2: Create and associate a user.

Procedure:

- a. Create a user and associate it with the corresponding role.

user_supply_manager: associate with **role_supply_admin** and configure a password of **Supply@2024**.

user_supply_auditor: configure a password of Supply@2024.

Screenshot requirements:

- a. Take a screenshot of the SQL statement for creating the user, and save it as **3-2-1 user_create**.

Subtask 3: Configure column-level data masking.

Procedure:

- a. Switch to the **db_supplychain** database and create the **vw_supplier_masked** view to configure column-level data masking for **user_supply_auditor**: When the **s_phone** column in the **public.supplier** table is queried, the column is displayed as **** (only temporarily hidden, without changing the original data in the table).
- b. Configure permission control (ensure that the masking takes effect): Forbid the audit user **user_supply_auditor** to directly access the original table (force the user to query the table through a view); grant only the audit user **user_supply_auditor** the permission to access the view; and ensure that the administrator **user_supply_manager** can directly access the original table (all permissions reserved).
- c. Use the meta commands to switch the user to **user_supply_auditor** and **user_supply_manager** and log in to the database to query the view, respectively. A maximum of two records is returned for each query.

Screenshot requirements:

- a. Take a screenshot of the SQL statement for creating the **vw_supplier_masked** view and the execution success message, and save it as **3-3-1 mask_policy**.
- b. Take a screenshot of the three SQL statements (one REVOKE and two GRANT statements) for permission configuration and the execution result, and save it as **3-3-2 mask_control**.
- c. Take a screenshot of the view query results (a maximum of two records for each) as **user_supply_auditor** and **user_supply_manager**, and save it as **3-3-3 mask_verify**.

Task 4: Supply Chain Performance Tuning and Comprehensive O&M (113 Points)

Subtask 1: Optimize SQL queries.

Environment preparation:

Connect to the **db_supplychain** database and create a new supply order table.

```
CREATE TABLE IF NOT EXISTS supply_order_new (  
    order_id INT PRIMARY KEY,           -- Order ID (PRIMARY KEY)  
    part_id INT NOT NULL,                -- Part ID (target part ID: 10086)  
    supplier_id INT NOT NULL,            -- Supplier ID  
    supply_qty INT NOT NULL,             -- Supply quantity
```



```

        supply_date TIMESTAMP NOT NULL,          -- Supply date (query range: 2024-01-01 to 2024-06-30)
        order_status VARCHAR(20) NOT NULL        -- Order status
        (DELIVERED/PENDING/CANCELLED)
    );
    -- Insert test data, including supply records of part 10086 in the last half year.
    INSERT INTO supply_order_new (order_id, part_id, supplier_id, supply_qty, supply_date, order_status)
    SELECT
        generate_series(1, 100000) AS order_id,  -- Unique order IDs (1–100,000)
        -- Control the record quantity (about 500) of part 10086 for future query and verification.
        CASE WHEN random() < 0.005 THEN 10086 ELSE floor(random()*10000 + 1) END AS part_id,
        floor(random()*800 + 1) AS supplier_id,  -- Random supplier ID (1–800)
        floor(random()*2000 + 10) AS supply_qty,  -- Supply quantity (10–2010, avoiding extreme values)
        -- Date distribution: 30% of the data was generated in the last half year (from 2024-01-01 to 2024-06-30), and 70% of the data is historical data.
        CASE
            WHEN random() < 0.3
            THEN '2024-01-01'::timestamp + (random()*181 || ' days')::interval  -- Last half year (181 days)
            ELSE '2023-01-01'::timestamp + (random()*365 || ' days')::interval  -- All days in 2023
        END AS supply_date,
        -- Random order status (even distribution)
        (ARRAY['DELIVERED', 'PENDING', 'CANCELLED'])[floor(random()*3 + 1)::INT] AS order_status;

```

Procedure:

- Collect statistics on the number of daily supply times of the part (**part_id = 10086**) from 2024-01-01 to 2024-06-30.
- Create a composite index (matching query conditions + grouping columns) and view the actual execution plan after the index is added.

Screenshot requirements:

- Take a screenshot of the number of daily supply times of the part (**part_id = 10086**) from 2024-01-01 to 2024-06-30, and save it as **4-1-1 original_1**.
- Take a screenshot of the created composite index (matching query conditions + grouping columns) and the actual execution plan after the index is added, and save it as **4-1-2_process**.

Subtask 2: Design and test the logistics information verification trigger.

Data preparation:

Connect to the **db_supplychain** database and create the **logistics_info** table to store logistics information. The table contains the following columns:

id: PRIMARY KEY (auto-increment)

order_id: order ID (NOT NULL)

logistics_no: logistics number (NOT NULL)

carrier: carrier company code (NOT NULL, only **SF**, **YTO**, **STO**, **ZTO**, and **YD** allowed)

create_time: creation time (default value: current time)

Run the following statements to create the table structure:

```
CREATE TABLE logistics_info (  
    id SERIAL PRIMARY KEY,  
    order_id INT NOT NULL,  
    logistics_no VARCHAR(50) NOT NULL,  
    carrier VARCHAR(10) NOT NULL CHECK (carrier IN ('SF','YTO','STO','ZTO','YD')),  
    create_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Procedure:

Create the **trigger_check_logistics_no** function (returning the TRIGGER type) to implement **logistics_no** verification.

- a. The logistics No. must start with the carrier company code, like **SF** or **YTO**. Otherwise, the following message is displayed: "Logistics No. error: The prefix % does not match the carrier company code % (which must start with %)."
- b. The length of the logistics No.'s main part (after the code) must range from 10 to 15 digits. Otherwise, the following message is displayed: "Logistics No. error: The length of the main part must range from 10 to 15 digits. The current length is % digits."
- c. The logistics No.'s main part must consist of only digits (0–9). Otherwise, the following message is displayed: "Logistics No. error: Only digits are allowed in the main part (non-digit characters are included)."
- d. The total length of the complete logistics No. = Length of the carrier company code + Length of the main body. For example, the length of **SF1234567890123** is 15 (2 + 13), and the length of **YTO9876543210** is 13 (3 + 10).
- e. Create the **trg_check_logistics_no** trigger to trigger the verification function before inserting and updating the **logistics_info** table.

After the creation is complete, test the trigger.

Screenshot requirements:

- a. Take a screenshot of the complete code and execution result (including the CREATE FUNCTION success message) of the **trigger_check_logistics_no** trigger function, and save it as **4-2-1 create_trigger_func**.
- b. Take a screenshot of the statement for creating the **trg_check_logistics_no** trigger and the execution result (including the CREATE TRIGGER success message), and save it as **4-2-2 create_trigger**.
- c. Execute the following two statements. Take a screenshot of the statements and execution results, and save it as **4-2-3 insert_valid_data**.

```
INSERT INTO logistics_info (order_id, logistics_no, carrier)
VALUES (1001, 'SF1234567890123', 'SF');
INSERT INTO logistics_info (order_id, logistics_no, carrier)
VALUES (1002, 'YTO9876543210', 'YTO');
```

- d. Take a screenshot of running the following three INSERT SQL statements and the corresponding error messages, and save it as **4-2-4 insert_invalid_data**.

```
INSERT INTO logistics_info (order_id, logistics_no, carrier)
VALUES (1003, 'STO12345678901', 'ZTO');
INSERT INTO logistics_info (order_id, logistics_no, carrier)
VALUES (1004, 'YD123456789', 'YD');
INSERT INTO logistics_info (order_id, logistics_no, carrier)
VALUES (1005, 'SF1234abc567890', 'SF');
```

- e. Take a screenshot of running the following statements for updating invalid data and the ERROR message, and save it as **4-2-5 update_invalid_data**.

```
UPDATE logistics_info
SET logistics_no = 'YTO9876abcd1234'
WHERE id = 2;
```

- f. Take a screenshot of the SQL statement for querying all information in **logistics_info table** and execution result, and save it as **4-2-6 final_data_query**.

Subtask 3: Create and execute a stored procedure.

Run the following statements in the **db_supplychain** database:

```
CREATE TABLE product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL
);
```

-- **inventory** table

```
CREATE TABLE inventory (
    inventory_id INT PRIMARY KEY,
    product_id INT REFERENCES product(product_id),
    quantity INT NOT NULL,
    status VARCHAR(20) DEFAULT 'NORMAL'
```



```
);
INSERT INTO product VALUES
(101, 'Laptop'),
(102, 'Wireless mouse'),
(103, 'Mechanical keyboard'),
(104, 'Monitor');
INSERT INTO inventory VALUES
(1, 101, 8, 'NORMAL'), -- quantity < 10
(2, 102, 30, 'NORMAL'), -- quantity < 50
(3, 103, 60, 'NORMAL'), -- No condition is met.
(4, 104, 5, 'NORMAL'), -- quantity < 10 and < 50
(5, 102, 45, 'NORMAL'), -- quantity < 50
(6, 101, 70, 'NORMAL'); -- No condition is met.
```

Procedure:

- Create a stored procedure **update_stock_status** to automatically change **status** of records whose **quantity** is less than 10 in the **inventory** table to **OUT_OF_STOCK** and count the number of updated rows.

Screenshot requirements:

- Take a screenshot of the stored procedure creation command and the success message displaying **Procedure created.**, and save it as **4-3_create_proc**.
Take a screenshot of comparing the status before and after the execution, **NORMAL** before the execution and **OUT_OF_STOCK** after the execution, and save it as **4-3_execute_proc**.

Subtask 4: Audit and trace operations.

Procedure:

The database administrator wants to check whether recent operations of user **stock_audit** in the **db_supplychain** database are compliant. Complete the following operations:

- Create the **stock_audit** user, set the password to **Audit@2025**, and grant the **CONNECT** permission on the **db_supplychain** database and the **UPDATE** permission on the **inventory** table in the database to the user.
- Log in to the **db_supplychain** database as the **stock_audit** user and run the following **UPDATE** statements:

```
UPDATE inventory SET quantity = 35 WHERE inventory_id = 2;
UPDATE inventory SET quantity = 50 WHERE inventory_id = 5;
```

- Query all operations performed by the **stock_audit** user on the **db_supplychain** database in the last 24 hours.

Screenshot requirements:

- Take a screenshot of the SQL statements for creating the user and granting permissions and the execution result, and save it as **4-4-1 audit_config**.
- Take a screenshot of the user switching and update success records, and save it as **4-4-2 audit_update**.
- Take a screenshot of all operations performed by the **stock_audit** user on the **db_supplychain** database in the last 24 hours, and save it as **4-4-3 audit_result**.

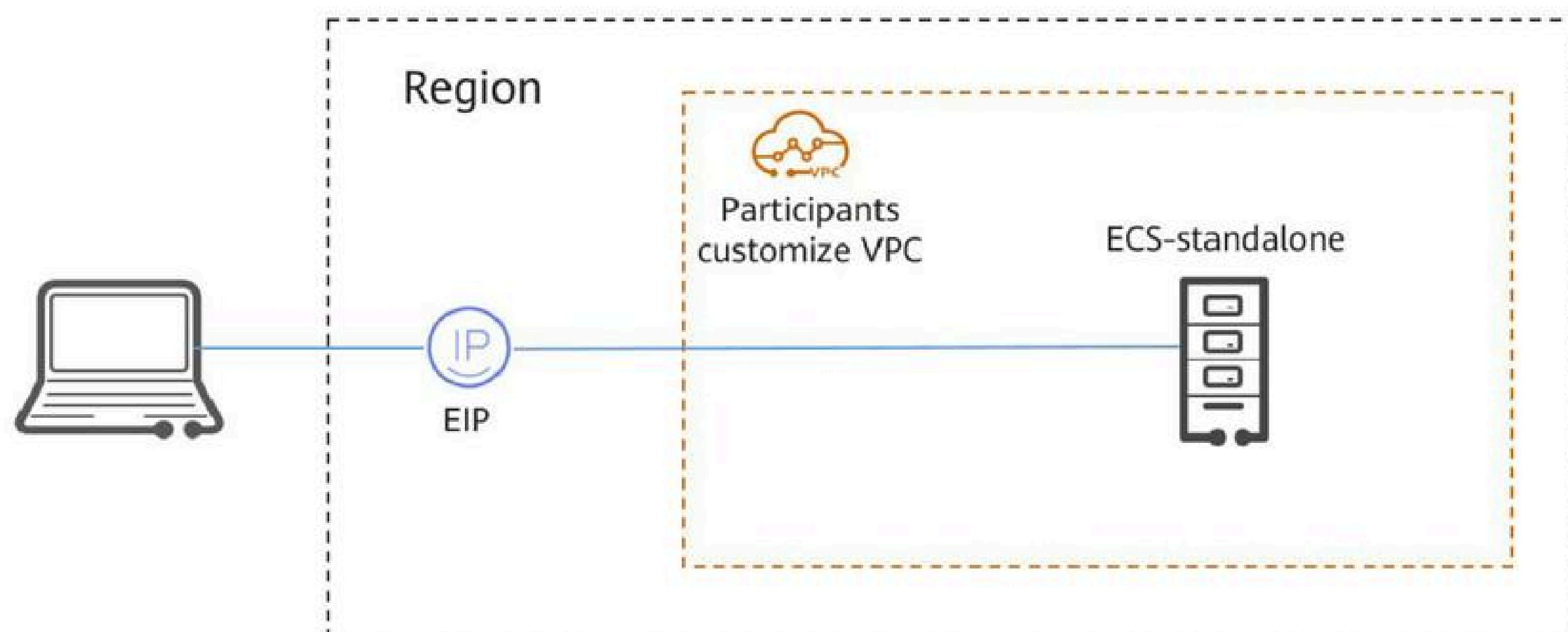
3.3 Kunpeng Application Development Tasks

3.3.1 Scenario

A company is porting its existing service system from x86 to the Kunpeng architecture so that it can utilize the high performance and power efficiency of Kunpeng processors. The service system comprises multiple modules, of which one key module uses spdlog, a fast logging library, for log management, and a function module needs to apply the high-performance Kunpeng Math Library (KML). To ensure smooth porting and performance optimization, the company decides to use the Kunpeng DevKit for cross-platform porting and application development based on the KML.

3.3.2 Network Topology

Description: The following figure shows the network topology of this exam. You can define a VPC or use the default VPC.



3.3.3 Exam Resources

3.3.3.1 Lab Environment

Set up the Huawei Cloud environment in accordance with the exam requirements. For accounts created on the Huawei Cloud international website, the CN-Hong Kong or AP-Singapore region is recommended. Your computer must have Internet access to complete the tasks in this exam.

3.3.3.2 Cloud Resources

Purchase and configure resources according to the following table, and set the names and passwords as required.

Resource Name	Specifications	Description
VPC	None	Use a custom VPC name or vpc-default .
ECS ecs-standalone	4 vCPUs 8 GiB openEuler 22.03 64bit with ARM Disk capacity: 60 GB	Use the Kunpeng architecture.
EIP and bandwidth	As required by the tasks	Use multiple EIPs.
IMS	Shared image name: ict-kunpeng	

3.3.3.3 Tools

The following table lists the tool required in this exam.

Tool	Description
MobaXterm	Remote login tool. Other remote login tools can also be used.

The following table lists the software or source packages to be downloaded for this exam.

Software Package	Description
DevKit-All-25.0.0-Linux-Kunpeng.tar.gz	Used to deploy the Kunpeng DevKit in Task 1.
spdlog-1.11.0.zip	Used to analyze the spdlog source package in Task 1.
kml-2.3.0-1.aarch64.rpm	Used to develop an application based on the KML in Task 2.
KML_BLAS_Project.zip	Used to develop an application based on the KML in Task 2.

Note: All software packages are stored in the **/home/kunpeng** directory of the ECS. If the software packages are missing, verify that the shared private image is used to create the ECS.

3.3.4 Exam Tasks

Each subtask in a task is scored separately. Please plan your time carefully.

Task 1: Porting Code to Kunpeng Using the Kunpeng DevKit (110 Points)

Subtask 1: Create a Kunpeng ECS.

Procedure:

Create a Kunpeng ECS in a specified region.

Note: You need to select the shared ICT-kunpeng image. If another image is used, the following subtasks cannot be completed. For details about ECS specifications requirements, see the table in "Cloud Resources."

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

Take a screenshot of the ECS configurations, including the required parameters, and save it as **1-1-1 buyecs**.

Subtask 2: Deploy the C++ compilation environment.

Procedure:

Install the g++ and CMake tools on the created ECS.

Screenshot requirements:

- Take a screenshot of the output on the terminal after the tools are installed, including the keyword "Complete", and save it as **1-2-1complete**.
- Take screenshots of queried g++ and CMake versions, and save them as **1-2-2g++** and **1-2-3cmake**, respectively.

Subtask 3: Install and deploy all Kunpeng DevKit plugins.

Procedure:

- Install the Kunpeng DevKit and all its plugins on the created ECS.

Note: If the error message "GPG check FAILED" is displayed, you are advised to solve the problem by modifying the **/etc/yum.repos.d/openEuler.repo** configuration file and use **DevKit-All-25.0.0-Linux-Kunpeng.tar.gz** for installation.

- Log in to the Kunpeng DevKit WebUI.

Note: If the message "Your connection is not private" is displayed, click **Advanced > Proceed to ******.

Screenshot requirements:

- Take a screenshot of the output on the terminal after the Kunpeng DevKit is installed, including the output message "The DevKit Web console is running", and save it as **1-3-1devkit**.

- b. Take a screenshot of the login page of the administrator **devadmin**, and save it as **1-3-2devadmin**.

Subtask 4: Analyze spdlog source code using the Kunpeng Porting Advisor.

Procedure:

- a. Create a source code porting task, upload spdlog source code to a specific path, and configure parameters as required.
- b. Perform analysis and obtain the source code porting analysis report.

Screenshot requirements:

- a. Take a screenshot of the task creation page, including all parameters except **Custom x86 Macro**, and save it as **1-4-1spdlog**.
- b. Take a screenshot of the source code porting analysis report, including all results, and save it as **1-4-2analysis**.

Subtask 5: Modify the source code, compile, and execute.

Procedure:

- a. Modify the source code as suggested and ensure that all code in red has been modified.
- b. Go to the directory where the source code is stored, compile the modified source code, and generate the target file.

Note: Header file libraries required for compilation can be installed using the installation packages.

- c. Check the generated target file and ensure that the compilation is complete.
- d. Go to the directory where the target file is stored, execute the target file, and ensure that the functions are normal.

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

- a. Take screenshots of the source code porting suggestions in red and the modified code, and save the screenshots as **1-5-1code** and **1-5-2codemod**, respectively.
- b. Take screenshots of error information and compilation completion, and save the screenshots as **1-5-3error** and **1-5-4output**, respectively.
- c. Take a screenshot of the generated target file on the CLI, including the complete file name, and save it as **1-5-5target**.
- d. Take a screenshot of the target file execution results, and save it as **1-5-6result**.

Task 2: Application Development Based on the KML (90 Points)

Subtask 1: Install the KML.

Procedure:

- a. Install the KML on the ECS.

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

- a. Take a screenshot of the installation result, and save it as **2-1-1kmlInstall**.

Subtask 2: Modify the CMakeList.txt file.

Procedure:

- a. Modify the **CMakeList.txt** file of KML_BLAS_Project to integrate the KML.

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

- a. Take a screenshot of the modifications made to the **CMakeList.txt** file, and save it as **2-2-1mod**.

Subtask 3: Compile the program.

Procedure:

- a. Compile the program.

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

- a. Take a screenshot of the complete compilation process, and save it as **2-3-1complie**.

Subtask 4: Execute the program and compare the results.

Procedure:

- a. Go to the directory where the target file is stored, execute the target file, and compare the results.

Screenshot requirements: (Take multiple screenshots if the results cannot be included in one, and suffix the screenshot names with (1), (2), etc.)

- a. Take a screenshot of the target file execution results, and save it as **2-4-1result**.